

On the Hardness of Proving a ZX-Diagram Equality

Mathis Hage, *MSc Student at EPFL, Intern at Geneva University*

Abstract

The abstract goes here.

Index Terms

ZX-Calculus, Proof Complexity, Circuit Obfuscation.

I. INTRODUCTION

IN the last decades, the *ZX-Calculus* language for diagrammatic reasoning on qubits has received a lot of attention for the possibilities it offers in quantum computing. For example, one can use this language to reduce the T-gate count in a quantum circuit. Over the years, the theory behind the ZX-Calculus was hence refined, up to achieving a *complete* form of the language [1].

However, a lot of important matters linked to the ZX-Calculus are hard. The most famous one is the circuit extraction problem, i.e. the problem consisting of extracting a quantum circuit from a ZX-Diagram. Even though some efficient algorithms exist in some specific cases, for example with diagrams presenting mathematical structures called *flows* (see for example [2]), circuit extraction remains #P-complete in the general case [3].

In this case study, we look at another aspect of the ZX-Calculus. We said earlier that this language is *complete* : this means that two diagrams are equal if and only if there is a sequence of rules (allowed by the language) that transforming one diagram into the other. Therefore it is always possible to go from one diagram to another if they represent the same linear map on qubits. An interesting question can now be asked : what is the *number of steps* needed to prove an equality between two diagrams ? More specifically, let the language :

$\text{ZX-REWRITE} := \{(D_1, D_2, k) \mid D_1, D_2 \text{ are ZX-diagrams s.t. the equality } D_1 = D_2 \text{ can be proven true in at most } k \text{ steps}\}.$

What is its complexity class ? This problem not only stems from ZX-calculus theory, but also from a general proof complexity setting.

II. REQUIREMENTS

A. ZX-Calculus

This part is largely based on *ZX-calculus for the working quantum computer scientist* p. 1-34. [4]. We avoid redundant citation in the rest of this section.

ZX-calculus is a graphical way to represent and reason about linear maps between qubits. A ZX-diagram represents qubits flowing through wires, hitting some nodes, which are linear maps acting on them, and at some point coming out. Such a diagram is composed of multiple instances of a specific set of *generators*.

1) *Generators*: The first generators are the *spiders*. These are nodes that can be green or red, and take a parameter α . As in [4], our figures use grey spiders instead of red ones, and white spiders instead of green ones. A green (white) node with n input wires (i.e. connected to the left of the node), m output wires (i.e. connected to the right of the node) and parameter α , as shown in fig. (1),

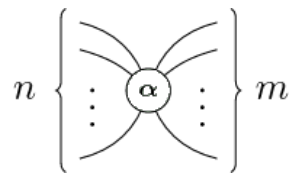


Fig. 1: Green Spider with n inputs, m outputs, and parameter α (taken from [5])

corresponds to the linear map :

$$\overbrace{|0 \dots 0\rangle}^m \overbrace{\langle 0 \dots 0|}^n + e^{i\alpha} \overbrace{|1 \dots 1\rangle}^m \overbrace{\langle 1 \dots 1|}^n.$$

If the node is red (grey), it corresponds to the linear map :

$$\overbrace{|+\dots+\rangle}^m \overbrace{\langle +\dots+|}^n + e^{i\alpha} \overbrace{|-\dots-\rangle}^m \overbrace{\langle -\dots-|}^n.$$

Here are a few examples :

- A green spider with 0 inputs and 1 output : $|0\rangle + e^{i\alpha} |1\rangle$;
- The same spider but red and parameter $\alpha = 0$ corresponds to $|0\rangle$, while with $\alpha = \pi$ it corresponds to $|1\rangle$.
- A green spider with one input and one output corresponds to the rotation matrix (on the Bloch sphere) by an angle α around the Z axis :

$$R_Z(\alpha) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\alpha} \end{pmatrix}$$

There are also other generators. First, the SWAP gate in fig. (2) (from quantum computing theory), simply represented as crossing wires.

$$\begin{array}{c} \text{X} \\ \text{= } \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{array} \qquad \begin{array}{c} \text{C} \\ \text{= } \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix} \end{array} \qquad \begin{array}{c} \text{) } \\ \text{= } \begin{pmatrix} 1 & 0 & 0 & 1 \end{pmatrix} \end{array}$$

(a) SWAP generator.

(b) The Cup and Cap generators

Fig. 2: The SWAP (a) and the cup and cap (b) generators.

One can freely slide spiders along these crossing wires. It is clear from this visual representation that SWAP is self-inverse : try composing two SWAP's side by side, you will notice that both qubits end up in their original lane.

Moreover, two other generators are of fundamental importance, namely the *cup* and the *cap* (respectively) drawn in fig. (2). Notice how they represent the Bell state and the Bell effect. Here also, a spider can freely move on a cup or a cap.

2) *Only connectivity matters*: Because of the fact that the spiders are symmetric, and using the SWAP, cups and caps generators, we arrive at the point that makes ZX-calculus very interesting : this can be summarized by the sentence "*only connectivity matters*". In fact, we can bend wires in any way we like, see them either as inputs or outputs of spiders, this doesn't change the overall linear map represented by the ZX-diagram as long as we don't change the order of the inputs and outputs of the whole diagram.

ZX-calculus is universal, meaning it can represent any linear map.

3) *Rules*: When working with ZX-diagrams, and in particular when trying to simplify them, one can make use of a set of rules to do so. These rules are summarized in fig. (3). These rules do not change the linear map when used.

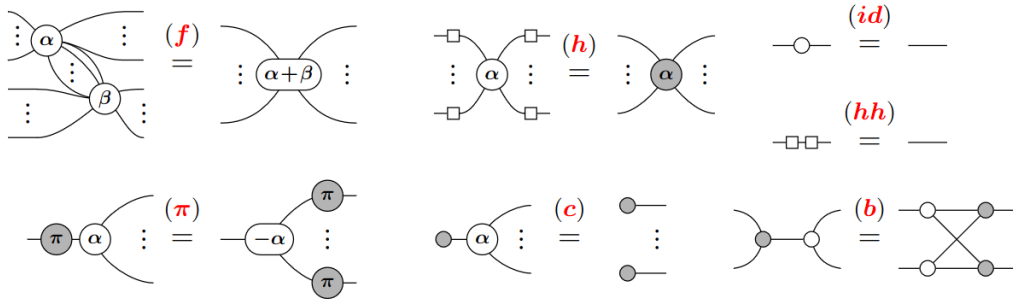


Fig. 3: The rules of ZX-calculus, namely : *spider fusion* (f), *hadamard* (h, hh), *identity* (id), *commute* (π), *copy* (c), *bialgebra* (b). Note that the square box represents the hadamard unitary, which can be built from spiders as well.

There also is a set of practical "meta"-rules, which can be derived from the above ones. While we do not detail them all, we simply mention that the rules of fig. (3) all hold with the colors interchanged, and with the inputs / outputs interchanged.

Note that we work with unnormalized quantum states, which is not an issue.

III. RESULTS

Now that the prerequisites have been introduced, we can dive into the heart of the problem. Intuitively, even by knowing that two diagrams represent the same linear map, the problem of finding a sequence of ZX-rules of length at most k to go from one to the other seems like a very difficult task. Even though a verifier running in polytime can be found relatively easily, hence proving NP-membership of the language ZX-REWRITE, it is much more complicated to prove the suspected NP-hardness. In fact, unlike problems that require checking if two diagrams are equal for example, we have to focus on the *topology* of the diagrams rather than on the transformation they represent.

- A. NP-membership
- B. NP-hardness

When α is arbitrary, we prove NP-hardness by a reduction from SUBSET-SUM. We start by reminding its definition.

Definition 1. SUBSET-SUM

$$\text{SUBSET-SUM} := \{(S, t) \mid S \subset \mathbb{N} \text{ is finite, } t \in \mathbb{N}, \text{ and } \exists U \subseteq S : \sum_{s \in U} s = t\}.$$

Now, we claim that $\text{SUBSET-SUM} \leq_p \text{ZX-REWRITE}$.

Proof. Let the function $f : \text{SUBSET-SUM} \rightarrow \text{ZX-REWRITE}$ be defined as follows :

Algorithm 1 Reduction from SUBSET-SUM to ZX-REWRITE

```

1: function  $f(\langle S, t \rangle)$ 
2:   Let  $n = |S|$ 
3:   if  $t > \sum_{s \in S} s$  then
4:     Let  $D_1$  be a single 0-phase green spider with one input and one output.
5:     Let  $D_2$  be a wire containing two 0-phase green spiders.
6:     return  $\langle D_1, D_2, 0 \rangle$ 
7:   else
8:      $\mathcal{N} \leftarrow \sum_{s \in S} s$ 
9:     Let  $D_1$  consist of  $n$  green spiders disposed vertically, with the  $i^{\text{th}}$  spider having phase  $\frac{s_i}{\mathcal{N}}$  for  $s_i \in S$ .
10:    Connect the left side of each spider in  $D_1$  to a new 0-phase green spider (one wire each), and the right side to
    another 0-phase green spider (one wire each).
11:    Let  $D_2$  consist of two green spiders linked with  $n$  wires, where the left spider has phase  $\frac{\mathcal{N}-t}{\mathcal{N}}$  and the right spider
    has phase  $\frac{t}{\mathcal{N}}$ .
12:    return  $\langle D_1, D_2, n \rangle$ 
13:  end if
14: end function

```

Clearly, this algorithm runs in time $\mathcal{O}(n)$ where $n := |S|$, and hence f is a polynomial time computable function. Now, we need to show :

$$\langle S, t \rangle \in \text{SUBSET-SUM} \Leftrightarrow f(\langle S, t \rangle) = \langle D_1, D_2, k \rangle \in \text{ZX-REWRITE}.$$

Clearly if $t > \sum_{s \in S} s$, the reduction outputs a string that is not in ZX-REWRITE, as it is impossible to go from D_1 to D_2 in $k = 0$ steps (they are not equal).

Otherwise, $k = n$ and :

- ($\Rightarrow$) $\langle S, t \rangle \in \text{SUBSET-SUM} \implies \exists U \subseteq S$ s.t. $\sum_{s \in U} s = t$. Start with D_2 . By fusing every spider of the form $\frac{s}{\mathcal{N}}, s \in U$ with the right 0-phase spider, its phase becomes $\sum_{s \in U} \frac{s}{\mathcal{N}} = \frac{1}{\mathcal{N}} \sum_{s \in U} s = \frac{t}{\mathcal{N}}$. By fusing the remaining ones with the left 0-phase spider, its phase becomes $\sum_{s \notin U} \frac{s}{\mathcal{N}} = \frac{\mathcal{N}-t}{\mathcal{N}}$. Hence we have recovered D_2 , in n steps exactly.
- ($\Leftarrow$) It is possible to go from D_1 to D_2 in n steps. Consider such a sequence of n steps. As we go down from $n+2$ spiders to only 2, and that we only have green spiders, we can only use spider fusion : using another rule would cause us to "lose time" while we only have n moves allowed. Therefore, by creating a set U containing all $s \in S$ such that node $\frac{s}{\mathcal{N}}$ was slid to the right, we have that $\langle S, t \rangle \in \text{SUBSET-SUM}$.

Therefore, f is a valid reduction, which proves $\text{SUBSET-SUM} \leq_p \text{ZX-REWRITE}$. □

Corollary 1. ZX-REWRITE is NP-hard.

IV. CONCLUSION

The conclusion goes here.

ACKNOWLEDGMENT

REFERENCES

- [1] E. Jeandel, S. Perdrix, and R. Vilmart, "Completeness of the zx-calculus," *Logical Methods in Computer Science*, vol. Volume 16, Issue 2, 2020. [Online]. Available: [http://dx.doi.org/10.23638/LMCS-16\(2:11\)2020](http://dx.doi.org/10.23638/LMCS-16(2:11)2020)
- [2] A. Kissinger and J. van de Wetering, "Zx-flow: A flexible criterion for deterministic computation with zx-diagrams," 2026. [Online]. Available: <https://arxiv.org/abs/2603.09580>
- [3] M. Backens, H. Miller-Bakewell, G. de Felice, L. Lobski, and J. van de Wetering, "There and back again: A circuit extraction tale," *Quantum*, vol. 5, p. 421, Mar. 2021. [Online]. Available: <http://dx.doi.org/10.22331/q-2021-03-25-421>
- [4] J. van de Wetering, "Zx-calculus for the working quantum computer scientist," 2020. [Online]. Available: <https://arxiv.org/abs/2012.13966>
- [5] Wikipedia contributors, "Zx-calculus," <https://en.wikipedia.org/w/index.php?title=ZX-calculus>, 2026, [Online; accessed August 10, 2026].